



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 02

NOMBRE COMPLETO: VELARDE VALENCIA JOSUE

N° de Cuenta: 320342330

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 07

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 09/09/2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1. Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Para realizar la primer actividad utilicé de referencia las figuras previamente hechas en la función `void CrearLetrasyFiguras()` y los vértices de las letras creadas en la práctica anterior, solo copié las coordenadas de las letras y agregué 3 datos más a cada línea para los valores RGB, asignando a cada letra un color distinto.

```
void CrearLetrasyFiguras()
{
    GLfloat vertices_letraJ[] = {
        //X    Y    Z    R    G    B
        //Letra J
        -1.0f, 0.6f, 0.0f, 1.0f, 0.0f, 0.0f,
        -1.0f, 0.5f, 0.0f, 1.0f, 0.0f, 0.0f,
        -0.3f, 0.6f, 0.0f, 1.0f, 0.0f, 0.0f,
        -0.3f, 0.6f, 0.0f, 1.0f, 0.0f, 0.0f,
        -0.3f, 0.5f, 0.0f, 1.0f, 0.0f, 0.0f,
        -1.0f, 0.5f, 0.0f, 1.0f, 0.0f, 0.0f,
        -0.6f, 0.5f, 0.0f, 1.0f, 0.0f, 0.0f,
        -0.7f, 0.5f, 0.0f, 1.0f, 0.0f, 0.0f,
        -0.6f, -0.4f, 0.0f, 1.0f, 0.0f, 0.0f,
        -0.7f, 0.5f, 0.0f, 1.0f, 0.0f, 0.0f,
        -0.6f, -0.4f, 0.0f, 1.0f, 0.0f, 0.0f,
        -0.7f, -0.4f, 0.0f, 1.0f, 0.0f, 0.0f,
        -0.7f, -0.4f, 0.0f, 1.0f, 0.0f, 0.0f,
        -0.6f, -0.4f, 0.0f, 1.0f, 0.0f, 0.0f,
        -0.7f, -0.5f, 0.0f, 1.0f, 0.0f, 0.0f,
        -0.7f, -0.4f, 0.0f, 1.0f, 0.0f, 0.0f,
        -0.7f, -0.5f, 0.0f, 1.0f, 0.0f, 0.0f,
        -0.9f, -0.4f, 0.0f, 1.0f, 0.0f, 0.0f,
        -0.7f, -0.5f, 0.0f, 1.0f, 0.0f, 0.0f,
        -0.9f, -0.5f, 0.0f, 1.0f, 0.0f, 0.0f,
        -0.9f, -0.4f, 0.0f, 1.0f, 0.0f, 0.0f,
        -0.9f, -0.5f, 0.0f, 1.0f, 0.0f, 0.0f,
        -0.9f, -0.4f, 0.0f, 1.0f, 0.0f, 0.0f,
        -1.0f, -0.4f, 0.0f, 1.0f, 0.0f, 0.0f,
        -1.0f, -0.4f, 0.0f, 1.0f, 0.0f, 0.0f,
        -0.9f, -0.4f, 0.0f, 1.0f, 0.0f, 0.0f,
        -1.0f, -0.2f, 0.0f, 1.0f, 0.0f, 0.0f,
        -1.0f, -0.2f, 0.0f, 1.0f, 0.0f, 0.0f,
        -0.9f, -0.2f, 0.0f, 1.0f, 0.0f, 0.0f,
        -0.9f, -0.4f, 0.0f, 1.0f, 0.0f, 0.0f,
    };
    MeshColor* letraJ = new MeshColor();
    letraJ->CreateMeshColor(vertices_letraJ, 180);
    meshColorList.push_back(letraJ);
}
```

```

GLfloat vertices_letra0[] = {
    //X      Y      Z      R      G      B
    //Letra 0
    -0.1f,0.6f,0.0f, 1.0f, 0.0f, 0.5f,
    -0.1f,0.5f,0.0f, 1.0f, 0.0f, 0.5f,
    0.2f,0.6f,0.0f, 1.0f, 0.0f, 0.5f,
    0.2f,0.6f,0.0f, 1.0f, 0.0f, 0.5f,
    -0.1f,0.5f,0.0f, 1.0f, 0.0f, 0.5f,
    0.2f,0.5f,0.0f, 1.0f, 0.0f, 0.5f,
    -0.2f,0.5f,0.0f, 1.0f, 0.0f, 0.5f,
    -0.1f,0.5f,0.0f, 1.0f, 0.0f, 0.5f,
    -0.1f,0.6f,0.0f, 1.0f, 0.0f, 0.5f,
    0.3f,0.5f,0.0f, 1.0f, 0.0f, 0.5f,
    0.3f,-0.4f,0.0f, 1.0f, 0.0f, 0.5f,
    0.2f,0.5f,0.0f, 1.0f, 0.0f, 0.5f,
    0.2f,0.5f,0.0f, 1.0f, 0.0f, 0.5f,
    0.2f,-0.4f,0.0f, 1.0f, 0.0f, 0.5f,
    0.3f,-0.4f,0.0f, 1.0f, 0.0f, 0.5f,
    0.2f,0.5f,0.0f, 1.0f, 0.0f, 0.5f,
    0.2f,0.6f,0.0f, 1.0f, 0.0f, 0.5f,
    0.3f,0.5f,0.0f, 1.0f, 0.0f, 0.5f,
    -0.1f,0.5f,0.0f, 1.0f, 0.0f, 0.5f,
    -0.2f,0.5f,0.0f, 1.0f, 0.0f, 0.5f,
    -0.1f,-0.4f,0.0f, 1.0f, 0.0f, 0.5f,
    -0.2f,0.5f,0.0f, 1.0f, 0.0f, 0.5f,
    -0.1f,-0.4f,0.0f, 1.0f, 0.0f, 0.5f,
    -0.2f,-0.4f,0.0f, 1.0f, 0.0f, 0.5f,
    -0.2f,-0.4f,0.0f, 1.0f, 0.0f, 0.5f,
    -0.1f,-0.5f,0.0f, 1.0f, 0.0f, 0.5f,
    -0.1f,-0.4f,0.0f, 1.0f, 0.0f, 0.5f,
    -0.1f,-0.4f,0.0f, 1.0f, 0.0f, 0.5f,
    -0.1f,-0.5f,0.0f, 1.0f, 0.0f, 0.5f,
    0.2f,-0.4f,0.0f, 1.0f, 0.0f, 0.5f,
    0.2f,-0.4f,0.0f, 1.0f, 0.0f, 0.5f,
    0.2f,-0.5f,0.0f, 1.0f, 0.0f, 0.5f,
    -0.1f,-0.5f,0.0f, 1.0f, 0.0f, 0.5f,
    0.2f,-0.4f,0.0f, 1.0f, 0.0f, 0.5f,
    0.2f,-0.5f,0.0f, 1.0f, 0.0f, 0.5f,
    0.3f,-0.4f,0.0f, 1.0f, 0.0f, 0.5f
};

MeshColor* letra0 = new MeshColor();
letra0->CreateMeshColor(vertices_letra0, 216);
meshColorList.push_back(letra0);

```

```

GLfloat vertices_letraS[] = {
    //X      Y      Z      R      G      B
    //Letra S
    0.5f,0.6f,0.0f, 1.0f, 1.0f, 0.0f,
    0.5f,0.5f,0.0f, 1.0f, 1.0f, 0.0f,
    0.9f,0.6f,0.0f, 1.0f, 1.0f, 0.0f,
    0.9f,0.5f,0.0f, 1.0f, 1.0f, 0.0f,
    0.9f,0.6f,0.0f, 1.0f, 1.0f, 0.0f,
    0.5f,0.5f,0.0f, 1.0f, 1.0f, 0.0f,
    0.5f,0.6f,0.0f, 1.0f, 1.0f, 0.0f,
    0.4f,0.5f,0.0f, 1.0f, 1.0f, 0.0f,
    0.5f,0.5f,0.0f, 1.0f, 1.0f, 0.0f,
    0.5f,0.5f,0.0f, 1.0f, 1.0f, 0.0f,
    0.4f,0.5f,0.0f, 1.0f, 1.0f, 0.0f,
    0.5f,0.1f,0.0f, 1.0f, 1.0f, 0.0f,
    0.5f,0.1f,0.0f, 1.0f, 1.0f, 0.0f,
    0.4f,0.1f,0.0f, 1.0f, 1.0f, 0.0f,
    0.4f,0.5f,0.0f, 1.0f, 1.0f, 0.0f,
    0.5f,0.1f,0.0f, 1.0f, 1.0f, 0.0f,
    0.4f,0.1f,0.0f, 1.0f, 1.0f, 0.0f,
    0.5f,0.0f,0.0f, 1.0f, 1.0f, 0.0f,
    0.5f,0.0f,0.0f, 1.0f, 1.0f, 0.0f,
    0.5f,0.1f,0.0f, 1.0f, 1.0f, 0.0f,
    0.8f,0.1f,0.0f, 1.0f, 1.0f, 0.0f,
    0.8f,0.1f,0.0f, 1.0f, 1.0f, 0.0f,
    0.8f,0.0f,0.0f, 1.0f, 1.0f, 0.0f,
    0.5f,0.0f,0.0f, 1.0f, 1.0f, 0.0f,
    0.8f,0.1f,0.0f, 1.0f, 1.0f, 0.0f,
    0.8f,0.0f,0.0f, 1.0f, 1.0f, 0.0f,
    0.9f,0.0f,0.0f, 1.0f, 1.0f, 0.0f,
    0.9f,0.0f,0.0f, 1.0f, 1.0f, 0.0f,
    0.8f,0.0f,0.0f, 1.0f, 1.0f, 0.0f,
    0.9f,-0.4f,0.0f, 1.0f, 1.0f, 0.0f,
    0.8f,0.0f,0.0f, 1.0f, 1.0f, 0.0f,
    0.9f,-0.4f,0.0f, 1.0f, 1.0f, 0.0f,
    0.8f,-0.4f,0.0f, 1.0f, 1.0f, 0.0f,
    0.9f,-0.4f,0.0f, 1.0f, 1.0f, 0.0f,
    0.8f,-0.4f,0.0f, 1.0f, 1.0f, 0.0f,
    0.8f,-0.5f,0.0f, 1.0f, 1.0f, 0.0f,
    0.8f,-0.4f,0.0f, 1.0f, 1.0f, 0.0f,
    0.8f,-0.5f,0.0f, 1.0f, 1.0f, 0.0f,
    0.4f,-0.5f,0.0f, 1.0f, 1.0f, 0.0f,
    0.8f,-0.4f,0.0f, 1.0f, 1.0f, 0.0f,
    0.4f,-0.4f,0.0f, 1.0f, 1.0f, 0.0f,
    0.4f,-0.5f,0.0f, 1.0f, 1.0f, 0.0f
};

MeshColor* letraS = new MeshColor();
letraS->CreateMeshColor(vertices_letraS, 252);
meshColorList.push_back(letraS);

```

Para mostrar el resultado utilicé el *shaderColor*, el resultado se muestra a continuación:

```
// Shader para las letras
shaderList[1].useShader();

uniformModel = shaderList[1].getModelLocation();
uniformProjection = shaderList[1].getProjectLocation();

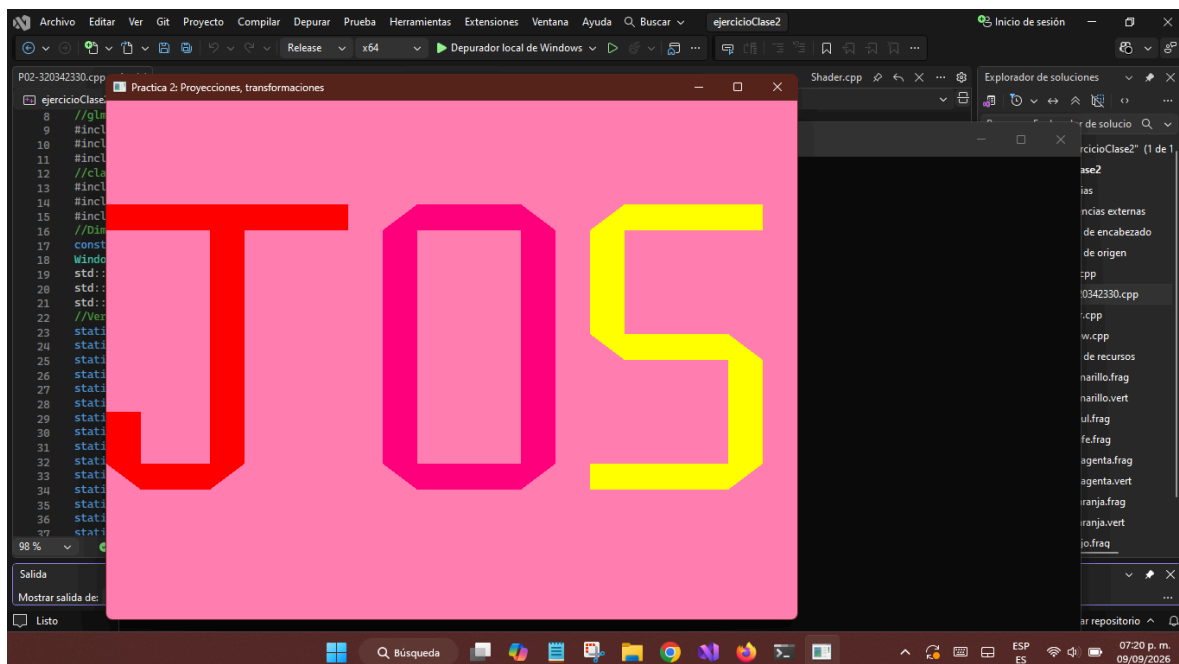
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -4.0f));

glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));

// Dibujar J
meshColorList[0]-->RenderMeshColor();

// Dibujar O
meshColorList[1]-->RenderMeshColor();

// Dibujar S
meshColorList[2]-->RenderMeshColor();
```



Para el segundo ejercicio, era necesario realizar el dibujo del ejercicio en clase utilizando las figuras 3D incluidas en el main proporcionado por el profesor y creando shaders individuales para cada color necesario. Para esto se deben crear 2 archivos de cada shader, un .vert y un .frag con el siguiente formato:

```
shaderamarillo.frag ✕ P02-320342330.cpp
1  #version 330
2  in vec4 vColor;
3  out vec4 color;
4  void main()
5  {
6      color = vColor;
7  }
```

```
shaderamarillo.vert ✕ P02-320342330.cpp
1  #version 330
2  layout(location = 0) in vec3 pos;
3  out vec4 vColor;
4  uniform mat4 model;
5  uniform mat4 projection;
6  void main()
7  {
8      gl_Position = projection * model * vec4(pos, 1.0f);
9      vColor = vec4(1.0f, 1.0f, 0.0f, 1.0f);
10 }
```

Deben agregarse los archivos de shaders de la siguiente manera:

```
//Vertex Shader
static const char* vShader = "shaders/shader.vert";
static const char* fShader = "shaders/shader.frag";
static const char* vShaderColor = "shaders/shadercolor.vert";
static const char* fShaderColor = "shaders/shadercolor.frag";
static const char* vShaderAmarillo = "shaders/shaderamarillo.vert";
static const char* fShaderAmarillo = "shaders/shaderamarillo.frag";
static const char* vShaderRojo = "shaders/shaderrojo.vert";
static const char* fShaderRojo = "shaders/shaderrojo.frag";
static const char* vShaderVerde = "shaders/shaderverde.vert";
static const char* fShaderVerde = "shaders/shaderverde.frag";
static const char* vShaderCafe = "shaders/shadercafe.vert";
static const char* fShaderCafe = "shaders/shadercafe.frag";
static const char* vShaderMagenta = "shaders/shadermagenta.vert";
static const char* fShaderMagenta = "shaders/shadermagenta.frag";
static const char* vShaderAzul = "shaders/shaderazul.vert";
static const char* fShaderAzul = "shaders/shaderazul.frag";
static const char* vShaderNaranja = "shaders/shadernaranja.vert";
static const char* fShaderNaranja = "shaders/shadernaranja.frag";
```

Posteriormente, se crean en la función *void createShaders()* y se agregan a la lista *shaderList()*:

```

void CreateShaders()
{
    Shader* shader1 = new Shader(); //shader para usar índices: o
    shader1->CreateFromFiles(vShader, fShader);
    shaderList.push_back(*shader1);
    Shader* shader2 = new Shader(); //shader para usar color como
    shader2->CreateFromFiles(vShaderColor, fShaderColor);
    shaderList.push_back(*shader2);
    Shader* shader3 = new Shader();
    shader3->CreateFromFiles(vShaderAmarillo, fShaderAmarillo);
    shaderList.push_back(*shader3);
    Shader* shader4 = new Shader();
    shader4->CreateFromFiles(vShaderRojo, fShaderRojo);
    shaderList.push_back(*shader4);
    Shader* shader5 = new Shader();
    shader5->CreateFromFiles(vShaderVerde, fShaderVerde);
    shaderList.push_back(*shader5);
    Shader* shader6 = new Shader();
    shader6->CreateFromFiles(vShaderCafe, fShaderCafe);
    shaderList.push_back(*shader6);
    Shader* shader7 = new Shader();
    shader7->CreateFromFiles(vShaderMagenta, fShaderMagenta);
    shaderList.push_back(*shader7);
    Shader* shader8 = new Shader();
    shader8->CreateFromFiles(vShaderAzul, fShaderAzul);
    shaderList.push_back(*shader8);
    Shader* shader9 = new Shader();
    shader9->CreateFromFiles(vShaderNaranja, fShaderNaranja);
    shaderList.push_back(*shader9);
}

```

Finalmente, hago el dibujo utilizando una proyección ortogonal, primero usé 3 pirámides triangulares para formar las partes de la primer figura aplicando una rotación de 180 grados sobre el eje X para cambiar su orientación. Usé dos cubos para los dos pilares cafés para completar la figura 1.

```

//Shader Amarillo
shaderList[2].useShader();
uniformModel = shaderList[2].getModelLocation();
uniformProjection = shaderList[2].getProjectLocation();
angulo += 0.01;
//Inicializar matriz de dimensión 4x4 que servirá como matriz de modelo para alm
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(-0.775f, 0.3f, -3.0f));
model = glm::scale(model, glm::vec3(0.2f, 0.2f, 1.0f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
meshList[2]->RenderMesh();

```



```
//Shader Rojo
shaderList[3].useShader();
uniformModel = shaderList[3].getModelLocation();
uniformProjection = shaderList[3].getProjectLocation();
angulo += 0.01;
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(-0.775f, 0.1f, -3.0f));
model = glm::scale(model, glm::vec3(0.2f, 0.2f, 1.0f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
meshList[2]->RenderMesh();
```

```
//Shader Verde
shaderList[4].useShader();
uniformModel = shaderList[4].getModelLocation();
uniformProjection = shaderList[4].getProjectLocation();
angulo += 0.01;
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(-0.775f, -0.1f, -3.0f));
model = glm::scale(model, glm::vec3(0.2f, 0.2f, 1.0f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
meshList[2]->RenderMesh();
```

```
//Shader Cafe
shaderList[5].useShader();
uniformModel = shaderList[5].getModelLocation();
uniformProjection = shaderList[5].getProjectLocation();
angulo += 0.01;
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(-0.9f, 0.175f, -3.0f));
model = glm::scale(model, glm::vec3(0.05f, 0.76f, 1.0f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
meshList[1]->RenderMesh();

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(-0.65f, 0.175f, -3.0f));
model = glm::scale(model, glm::vec3(0.05f, 0.76f, 1.0f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES
meshList[1]->RenderMesh();
```

Para la figura dos utilicé cuatro pirámides cuadrangulares que formarían un cuadrado de 4 colores diferentes, para después agregar dos cubos, uno café y otro azul con una rotación de 45 en el eje Z, el cubo azul se ubicó un punto más lejos que el cubo café en el eje Z para que se atravesaran entre sí y pudiera verse el café dentro del azul.


```

model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(-0.16f, 0.12f, -3.0f));
model = glm::scale(model, glm::vec3(0.21f, 0.21f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh();

```

```

model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.06f, 0.12f, -3.0f));
model = glm::scale(model, glm::vec3(0.21f, 0.21f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh();

```

```

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.06f, -0.092f, -3.0f));
model = glm::scale(model, glm::vec3(0.215f, 0.215f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh();

```

```

//Shader Naranja
shaderList[8].useShader();
uniformModel = shaderList[8].getModelLocation();
uniformProjection = shaderList[8].getProjectLocation();
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(-0.158f, -0.092f, -3.0f));
model = glm::scale(model, glm::vec3(0.215f, 0.215f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
meshList[1]->RenderMesh();

```

```

//Para el cubo y las pirámides se usa el primer set de shaders con índice 0 en
//Shader Azul
shaderList[7].useShader();
uniformModel = shaderList[7].getModelLocation();
uniformProjection = shaderList[7].getProjectLocation();
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(-0.05f, 0.015f, -3.0f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 1.0f));
model = glm::rotate(model, 45 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
meshList[1]->RenderMesh();

```

```

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(-0.05f, 0.015f, -2.0f));
model = glm::scale(model, glm::vec3(0.1f, 0.1f, 1.0f));
model = glm::rotate(model, 45 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES
meshList[1]->RenderMesh();

```

Para la figura 3 utilicé 3 pirámides rectangulares sin rotar, y uno con una rotación de 180 en el eje X, y para la base usé un cubo magenta con una escala más grande en el eje X que en el Y.

```

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.65f, 0.25f, -3.0f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[0]->RenderMesh();

```

```

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.5f, -0.05f, -3.0f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[0]->RenderMesh();

```

```

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.8f, -0.05f, -3.0f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[0]->RenderMesh();

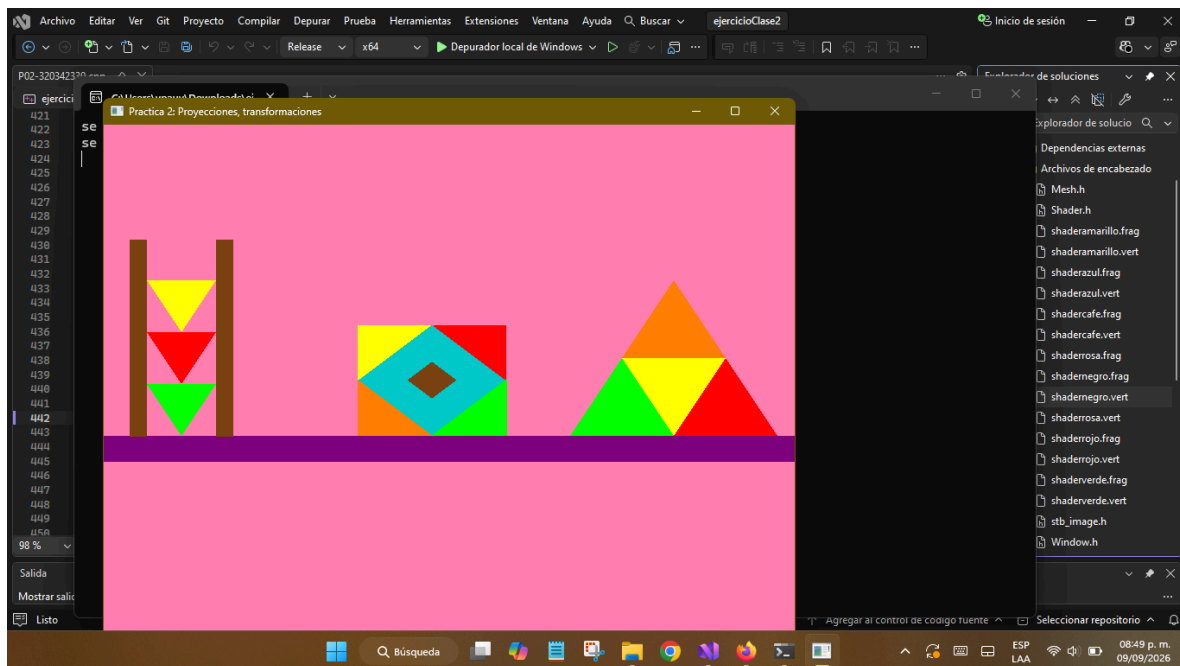
```

```

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.65f, -0.05f, -3.0f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 1.0f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[0]->RenderMesh();

```

La ejecución se muestra a continuación:



2. Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

No se presentó ningún problema al realizar la práctica, en esta ocasión pude desarrollar los ejercicios sin complicación.

3. Conclusión

Gracias a esta práctica logré entender el uso e implementación de shaders en los proyectos, que resultan ser muy útiles para generar diferentes figuras de un mismo color sin salir de dicho shader, además, las indicaciones fueron excelentes y entendí al instante lo solicitado para cada ejercicio.